

DOCUMENTACIÓN TÉCNICA

Carrito de Compras

Versión PHP y versión Spring Boot sobre GNU/Linux virtualizado

Dato	Información
Nombre del proyecto	Carrito de Compras (repositorio: Carrito)
Estudiante	Dafne Anely Ochoa Rustrian
Carné	3490-25-2461
Curso	Programación II
Docente	Ingeniero Moisés Velásquez
Fecha	17/10/2026
Versión del documento	1.0

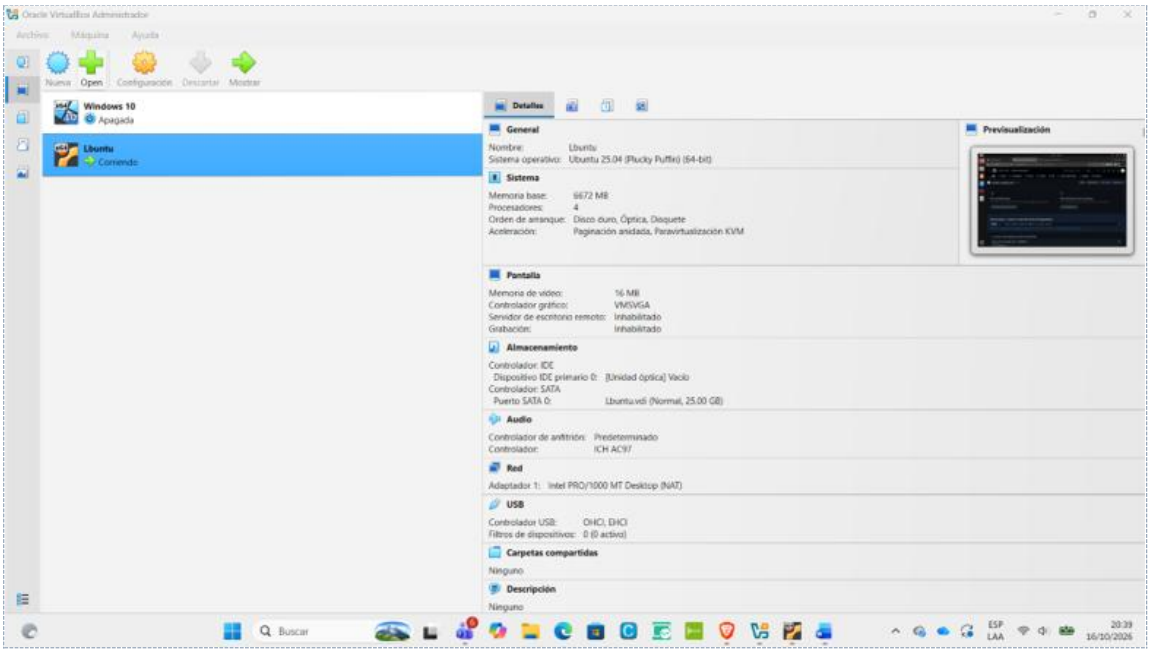
Contenido

2. Evidencia de virtualización 3
3. Justificación de recursos 5
4. Evidencia del servidor 6
5. Configuración del Virtual Host 9
6. Diagrama de casos de uso 10
7. Diagramas de arquitectura 11
8. Diccionario de datos 12
9. Contrato básico de la API 14
10. Evidencia de pruebas 15
11. Manual de usuario 16
12. Bitácora de desarrollo 17

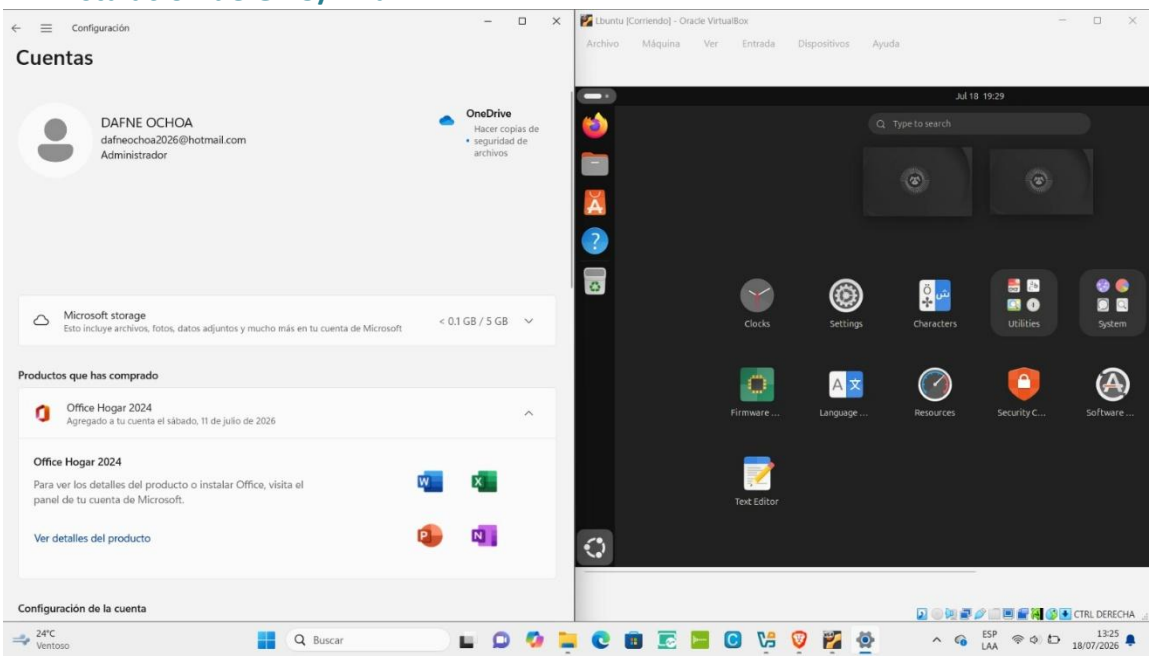
2. Evidencia de virtualización

Para el proyecto se utilizó VirtualBox como hipervisor sobre Windows, con el fin de ejecutar una máquina virtual con Ubuntu sin modificar el sistema anfitrión. Esto permite que el entorno de servidor Linux funcione de forma simultánea con Windows.

2.1 Creación de la máquina virtual

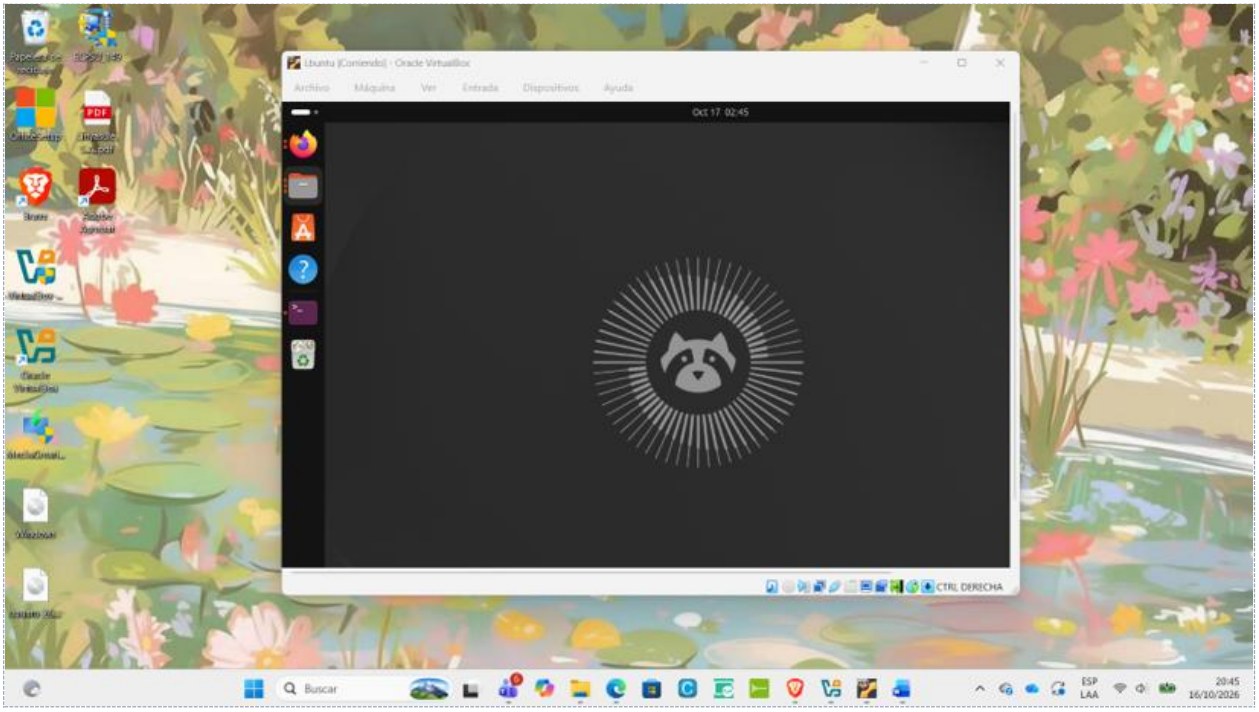


2.2 Instalación de GNU/Linux



2.3 Funcionamiento simultáneo con Windows

La siguiente evidencia muestra la máquina virtual en ejecución mientras Windows continúa operando con normalidad (por ejemplo, la ventana de la VM junto al Administrador de tareas o al escritorio de Windows).



3. Justificación de recursos

Los recursos asignados a la máquina virtual se definieron buscando un equilibrio: suficientes para que Apache, PHP, MySQL/MariaDB y Spring Boot funcionen con estabilidad, sin comprometer el rendimiento de Windows como sistema anfitrión.

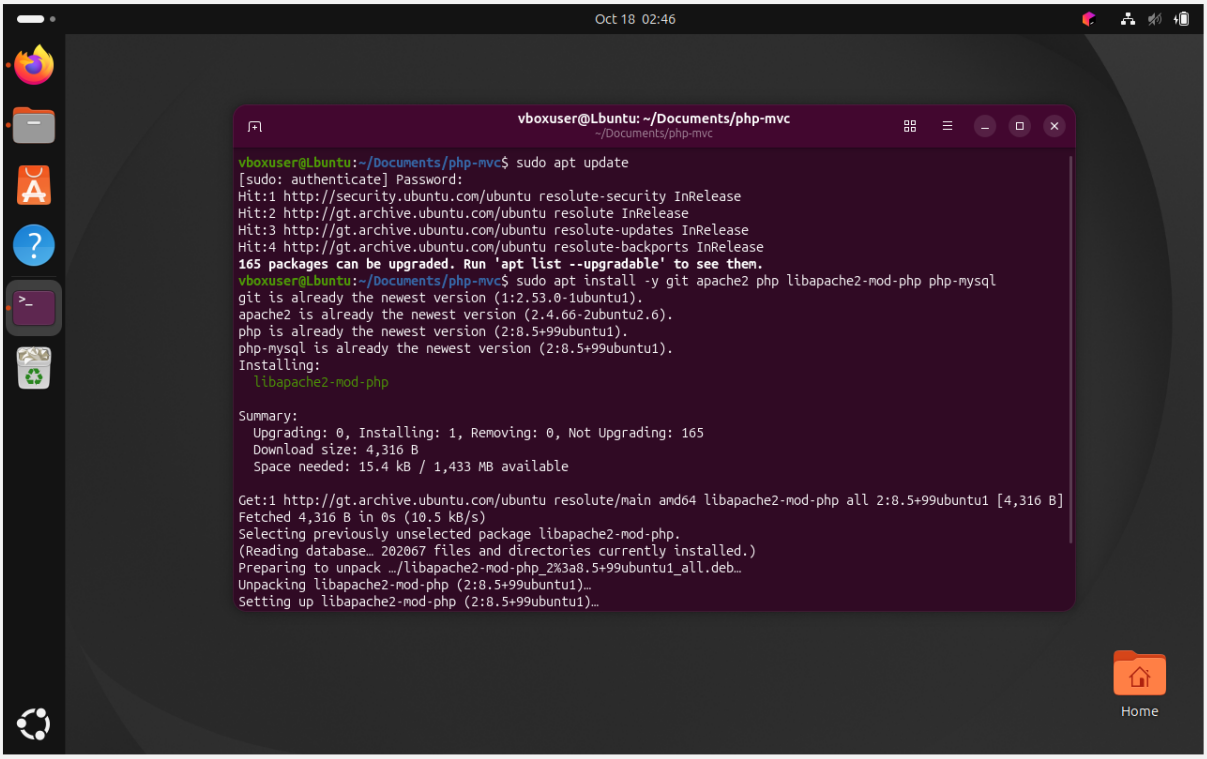
Recurso	Valor asignado	Justificación
Memoria RAM	6.5 GB	Apache + PHP + MariaDB requieren poca memoria; la JVM de Spring Boot es la que más consume (se recomienda al menos 1 GB para ella). El resto de la RAM queda para Windows.
CPU	4 núcleos	Permite atender el servidor web, la base de datos y la compilación de Spring Boot en paralelo sin saturar el procesador del anfitrión.
Almacenamiento	Lbuntu.vdi, SATA/AHCI	Cubre el sistema operativo, los paquetes del servidor, el repositorio Git, las dependencias de Maven/Gradle y la base de datos. El disco dinámico no ocupa espacio hasta que se necesita.
Red	NAT	Aísla la VM de la red local y le da acceso a internet para instalar paquetes. Para abrir el proyecto desde Windows se necesita un reenvío de puertos (por ejemplo, 8080 del anfitrión al 80 de la VM).

4. Evidencia del servidor

Dentro de la máquina virtual se instalaron Git, Apache, PHP y MySQL/MariaDB. A continuación, se muestran los comandos utilizados y las capturas que demuestran que los servicios están activos.

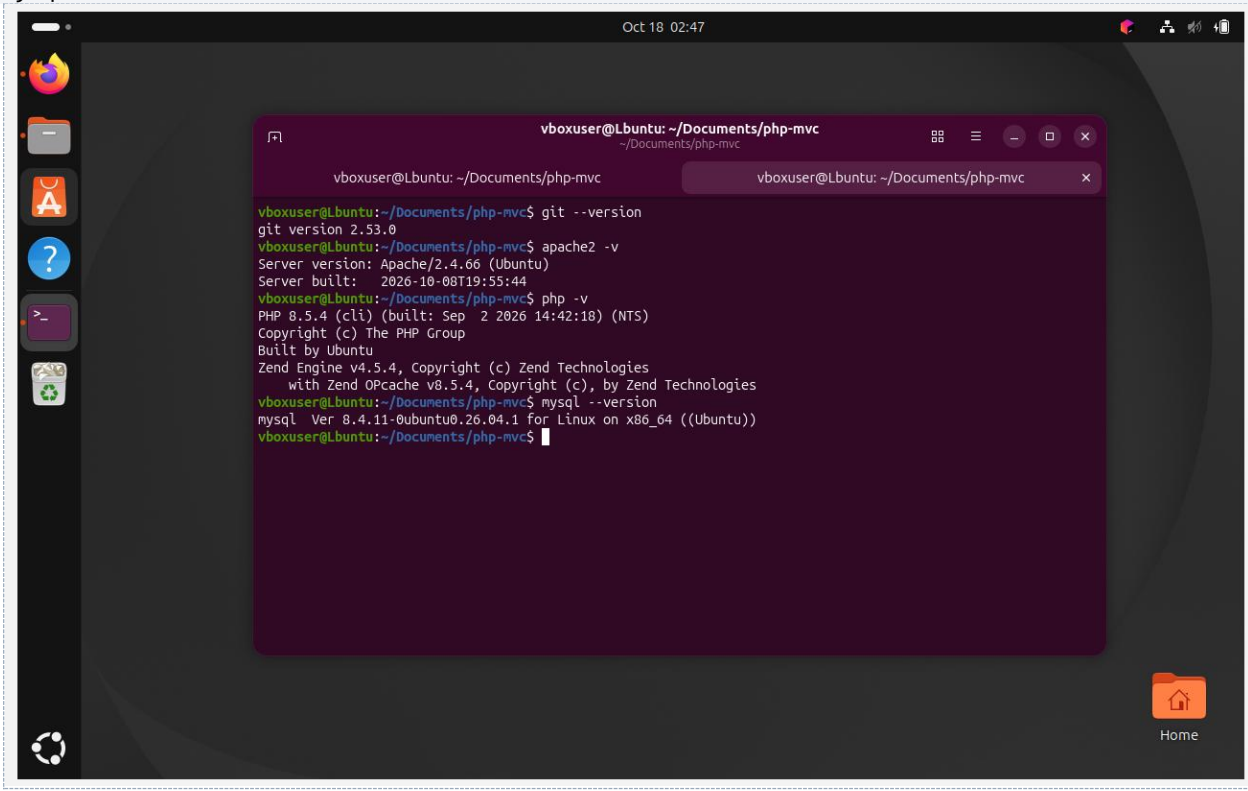
4.1 Instalación de paquetes

```
sudo apt update
sudo apt install -y git apache2 php libapache2-mod-php php-mysql mariadb-server
```



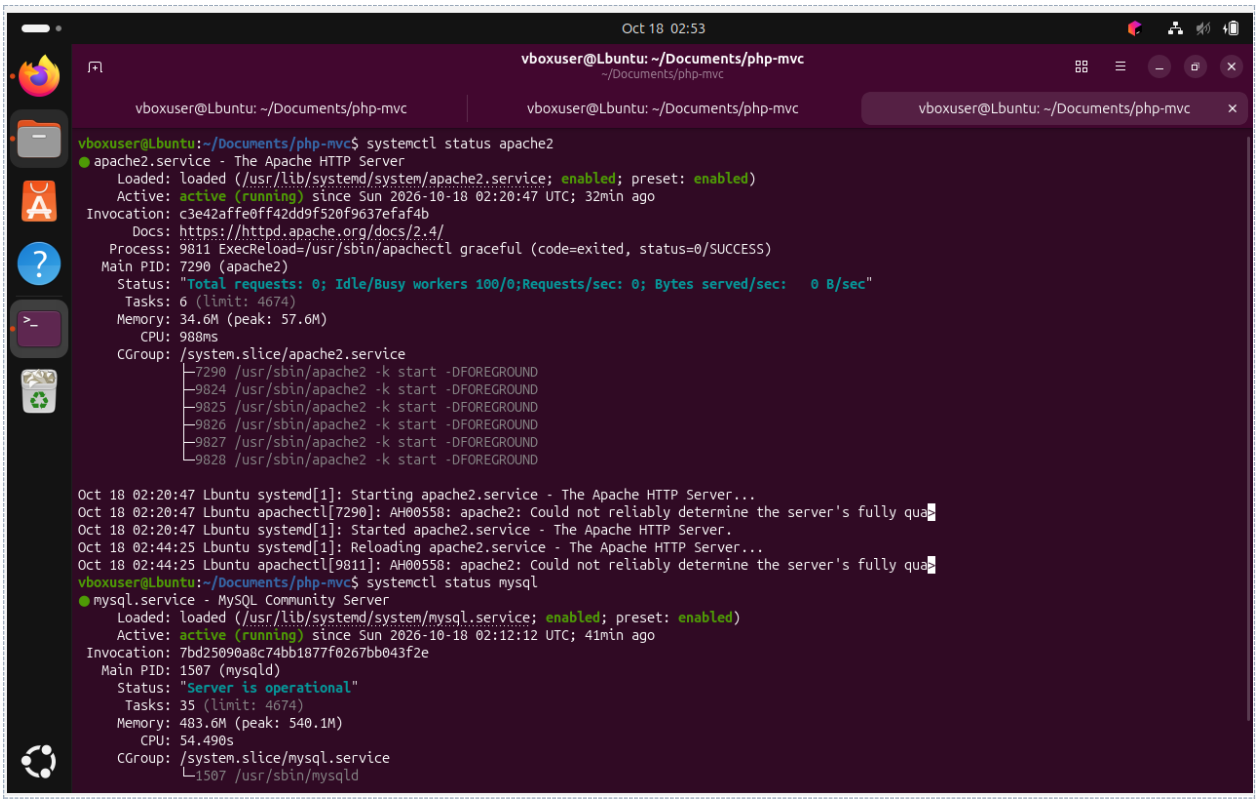
4.2 Verificación de versiones

```
git --version
apache2 -v
php -v
mysql --version
```



4.3 Servicios activos

```
systemctl status apache2
systemctl status mysql
```


Oct 18 02:53
vboxuser@Lbuntu: ~/Documents/php-mvc
~/Documents/php-mvc
vboxuser@Lbuntu: ~/Documents/php-mvc
vboxuser@Lbuntu: ~/Documents/php-mvc
vboxuser@Lbuntu: ~/Documents/php-mvc
vboxuser@Lbuntu:~/Documents/php-mvc$ systemctl status apache2
● apache2.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
Active: active (running) since Sun 2026-10-18 02:20:47 UTC; 32min ago
Invocation: c3e42affe0ff42dd9f520f9637efaf4b
Docs: https://httpd.apache.org/docs/2.4/
Process: 9811 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
Main PID: 7290 (apache2)
Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
Tasks: 6 (limit: 4674)
Memory: 34.6M (peak: 57.6M)
CPU: 988ms
CGroup: /system.slice/apache2.service
├─7290 /usr/sbin/apache2 -k start -DFOREGROUND
├─9824 /usr/sbin/apache2 -k start -DFOREGROUND
├─9825 /usr/sbin/apache2 -k start -DFOREGROUND
├─9826 /usr/sbin/apache2 -k start -DFOREGROUND
├─9827 /usr/sbin/apache2 -k start -DFOREGROUND
└─9828 /usr/sbin/apache2 -k start -DFOREGROUND
Oct 18 02:20:47 Lbuntu systemd[1]: Starting apache2.service - The Apache HTTP Server...
Oct 18 02:20:47 Lbuntu apachectl[7290]: AH00558: apache2: Could not reliably determine the server's fully qua>
Oct 18 02:20:47 Lbuntu systemd[1]: Started apache2.service - The Apache HTTP Server.
Oct 18 02:44:25 Lbuntu systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Oct 18 02:44:25 Lbuntu apachectl[9811]: AH00558: apache2: Could not reliably determine the server's fully qua>
vboxuser@Lbuntu:~/Documents/php-mvc$ systemctl status mysql
● mysql.service - MySQL Community Server
Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
Active: active (running) since Sun 2026-10-18 02:12:12 UTC; 41min ago
Invocation: 7bd25090a8c74bb1877f0267bb043f2e
Main PID: 1507 (mysqld)
Status: "Server is operational"
Tasks: 35 (limit: 4674)
Memory: 483.6M (peak: 540.1M)
CPU: 54.490s
CGroup: /system.slice/mysql.service
└─1507 /usr/sbin/mysqld

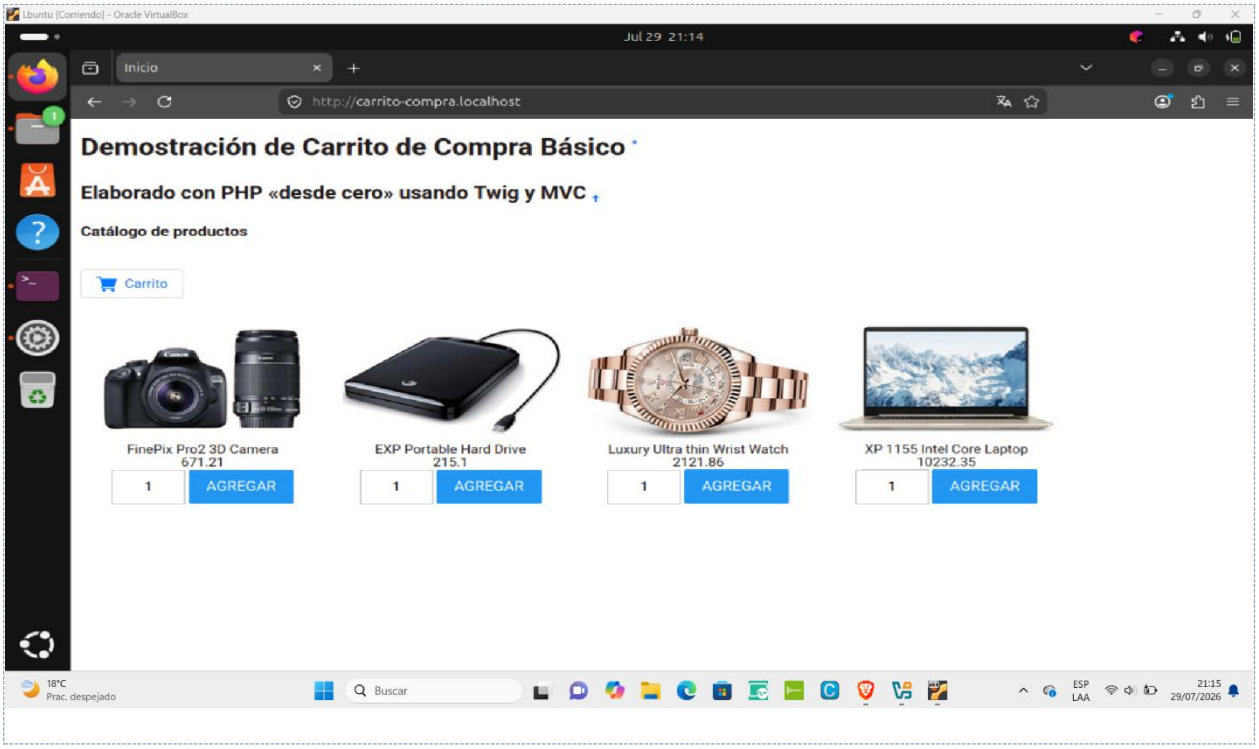
5. Configuración del Virtual Host

Se configuró un Virtual Host de Apache para servir el proyecto desde su propio nombre en lugar de la ruta por defecto. La dirección utilizada para acceder al proyecto es https://carrito-compra.localhost

5.1 Archivos modificados

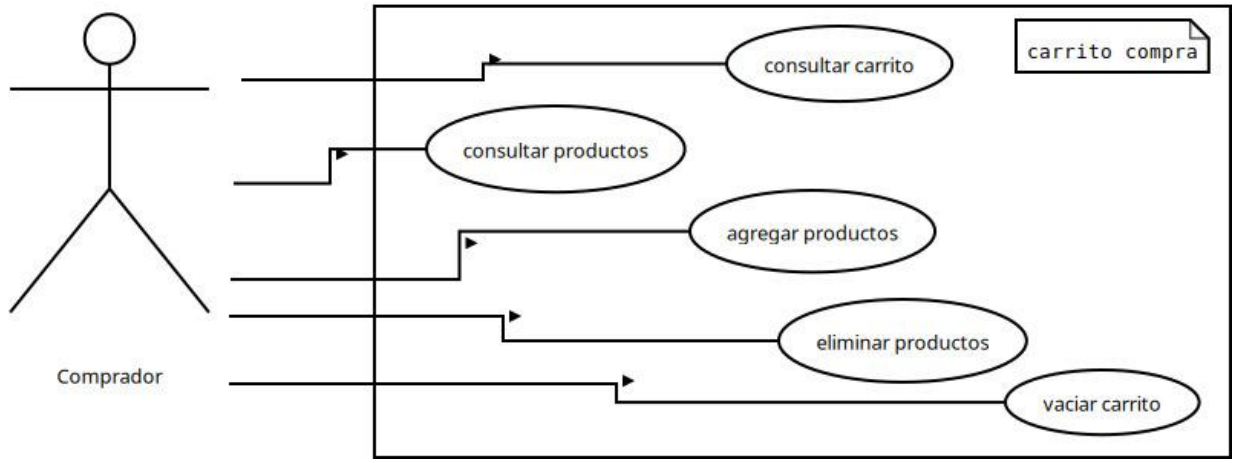
Archivo	Propósito
/etc/apache2/sites-available/carrito.conf	Define el VirtualHost: puerto, ServerName, DocumentRoot y permisos del directorio.
/etc/hosts (Linux) o C:\Windows\System32\drivers\etc\hosts	Asocia el nombre carrito compra con la IP de la máquina virtual para poder abrirlo desde el navegador.

5.3 Prueba de acceso al proyecto



6. Diagrama de casos de uso

El sistema tiene un único actor principal, el Cliente, quien interactúa con las tres funciones mínimas del carrito de compras.



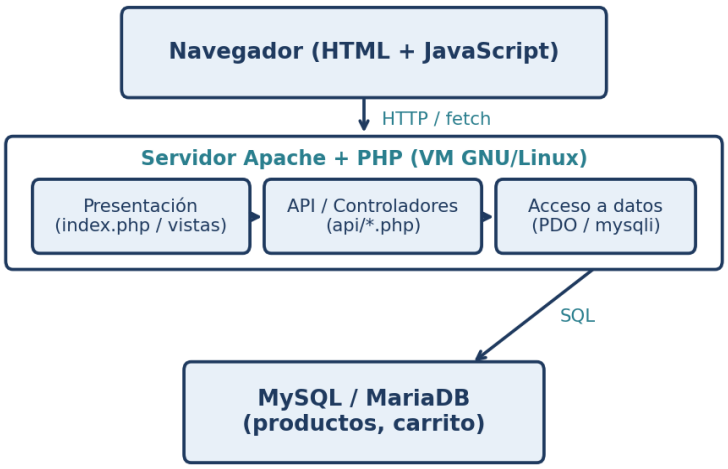
Caso de uso	Descripción
Consultar carrito	El cliente consulta la lista de productos disponibles con su nombre, precio y descripción.
Agregar productos	El cliente selecciona un producto y lo añade al carrito indicando la cantidad.
Eliminar productos	El cliente quita un producto que ya no desea del carrito.
Consultar productos	El cliente consulta los productos que ya ha agregado al carrito.
Vaciar carrito	El cliente puede dejar vacío su carrito

7. Diagramas de arquitectura

El proyecto se implementó en dos versiones con el mismo modelo de datos y el mismo contrato de API: una en PHP y otra en Spring Boot. Ambas se representan por capas.

7.1 Versión PHP

El navegador consume la API mediante peticiones HTTP. Apache atiende la solicitud y PHP la procesa: la capa de presentación muestra la interfaz, los controladores de la API reciben y validan las peticiones, y la capa de acceso a datos ejecuta las consultas SQL contra MySQL/MariaDB.



7.2 Versión Spring Boot

La aplicación sigue la arquitectura en capas típica de Spring: el Controller expone los endpoints REST y devuelve JSON; el Service contiene la lógica de negocio del carrito; el Repository, basado en Spring Data JPA, se encarga de la persistencia; y las Entity representan las tablas de la base de datos.

8. Diccionario de datos

La base de datos cc contiene dos tablas relacionadas: productos y carrito.

8.1 Tabla productos

Campo	Tipo	Clave	Descripción
id	INT AUTO_INCREMENT	PK	Identificador único del producto.
nombre	VARCHAR(100) NOT NULL	—	Nombre del producto.
descripcion	VARCHAR(255)	—	Descripción breve del producto.
precio	DECIMAL(10,2) NOT NULL	—	Precio unitario.
imagen	VARCHAR(255)	—	Ruta o URL de la imagen (opcional).

8.2 Tabla carrito

Campo	Tipo	Clave	Descripción
id	INT AUTO_INCREMENT	PK	Identificador único de la línea del carrito.
producto_id	INT NOT NULL	FK	Referencia a productos.id.
cantidad	INT NOT NULL DEFAULT 1	—	Unidades del producto en el carrito.
fecha_agregado	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	—	Fecha y hora en que se agregó.

8.3 Relación

Un producto puede aparecer en muchas líneas del carrito, y cada línea del carrito pertenece a un solo producto. Es una relación de uno a muchos (1:N) entre productos y carrito, implementada con la llave foránea carrito.producto_id → productos.id.

```
CREATE TABLE productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255),
  precio DECIMAL(10,2) NOT NULL,
  imagen VARCHAR(255)
);

CREATE TABLE carrito (
  id INT AUTO_INCREMENT PRIMARY KEY,
  producto_id INT NOT NULL,
  cantidad INT NOT NULL DEFAULT 1,
  fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (producto_id) REFERENCES productos(id)
);
```

9. Contrato básico de la API

Ambas versiones (PHP y Spring Boot) exponen los mismos endpoints. Las respuestas se entregan en formato JSON. URL base:

Método	Endpoint	Propósito	Parámetros
GET	/api/productos	Listar todos los productos.	Ninguno.
GET	/api/carrito	Ver el contenido del carrito.	Ninguno.
POST	/api/carrito	Agregar un producto al carrito.	Cuerpo JSON: producto_id (int, obligatorio), cantidad (int, opcional, por defecto 1).
DELETE	/api/carrito/{id}	Eliminar una línea del carrito.	Ruta: id de la línea del carrito.

9.1 Ejemplos de respuesta

GET /api/productos

```
[
  { "id": 1, "nombre": "Teclado", "descripcion": "Teclado USB", "precio": 150.00 },
  { "id": 2, "nombre": "Mouse", "descripcion": "Mouse óptico", "precio": 80.00 }
]
```

POST /api/carrito

```
// Petición
{ "producto_id": 1, "cantidad": 2 }

// Respuesta (201 Created)
{ "id": 5, "producto_id": 1, "cantidad": 2, "mensaje": "Producto agregado al carrito" }
```

GET /api/carrito

```
[
  { "id": 5, "producto_id": 1, "nombre": "Teclado", "precio": 150.00, "cantidad": 2 }
]
```

DELETE /api/carrito/5

```
{ "mensaje": "Producto eliminado del carrito" }
```

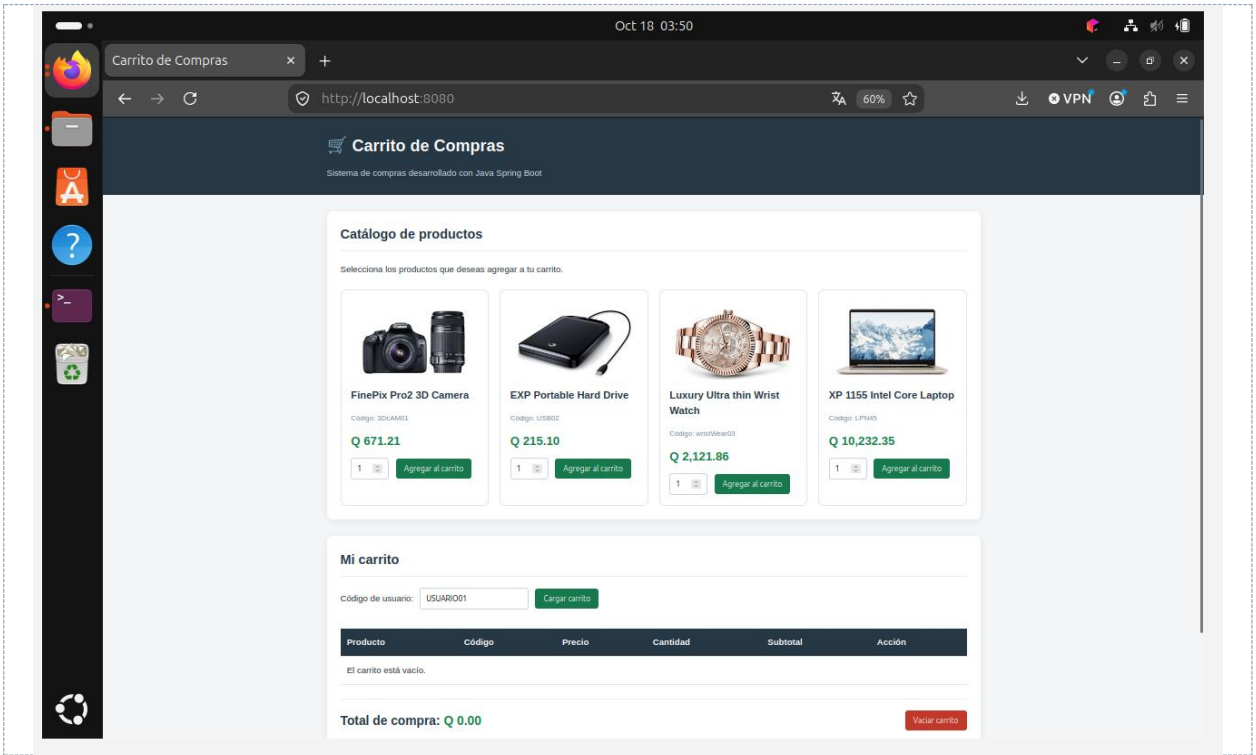
10. Evidencia de pruebas

Se probó cada endpoint de la API de forma individual con el navegador y después se verificó el flujo completo de extremo a extremo: frontend → API → base de datos → respuesta en pantalla.

10.1 Pruebas de la API

Prueba	Resultado esperado	Resultado obtenido	Estado
GET /api/productos	200 OK con lista de productos	Devolvió una lista JSON con 5 productos	OK
POST /api/carrito	201 Created con la línea agregada	Devolvió la línea agregada con id1	OK
GET /api/carrito	200 OK con los productos del carrito	Mostró el producto agregado con cantidad 2	OK
DELETE /api/carrito/{id}	200 OK y línea eliminada	El producto ya no aparece al consultar el carrito	OK

10.2 Prueba de integración de extremo a extremo



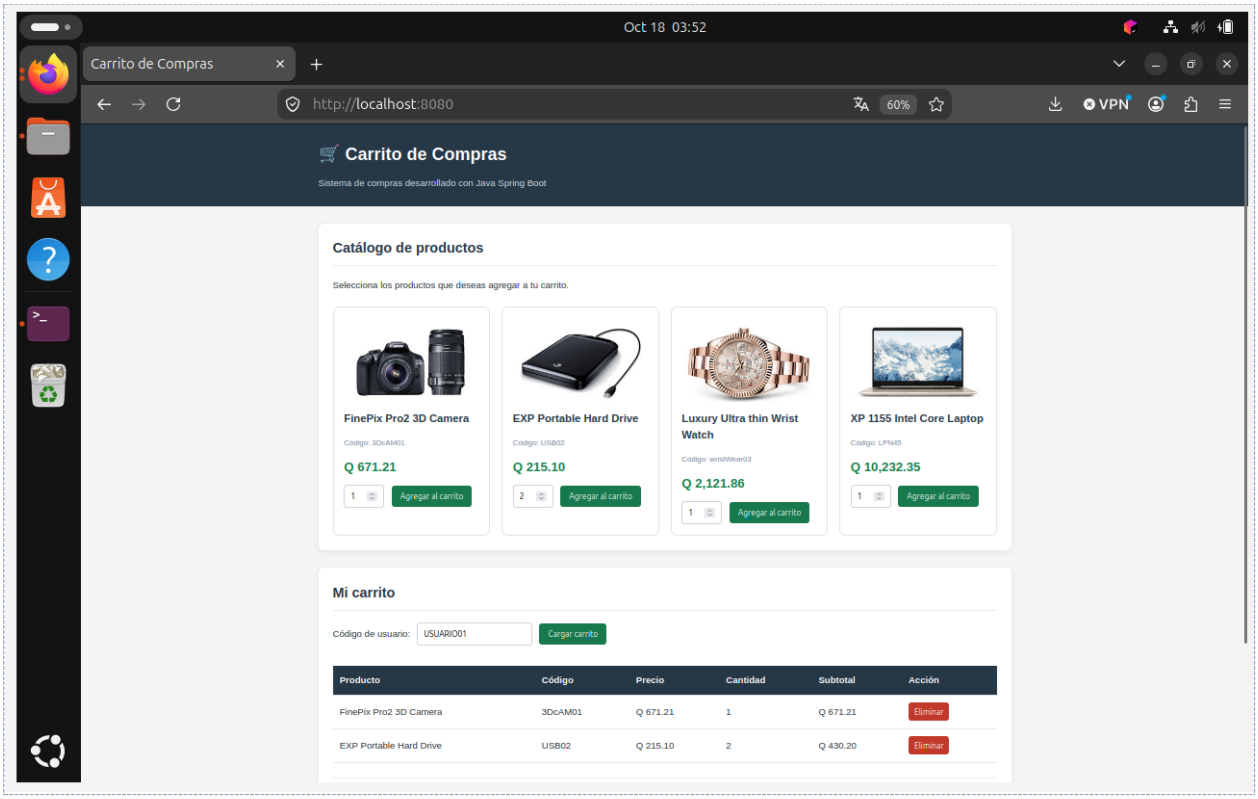
11. Manual de usuario

11.1 Iniciar el entorno

1. Abrir VirtualBox en Windows y encender la máquina virtual de Linux.
2. Iniciar sesión con el usuario Dafne Ochoa, contraseña Tito3003
3. Verificar que los servicios estén activos: sudo systemctl status apache2 y sudo systemctl status mariadb.
4. Versión PHP: abrir en el navegador de Windows la dirección https://carrito-compra.localhost
5. Versión Spring Boot: desde la carpeta del proyecto, ejecutar mvnw spring-boot:run y abrir http://localhost:8080

11.2 Usar el carrito

- Ver productos: al abrir la página principal se muestra la lista de productos con nombre, precio y descripción.
- Agregar al carrito: presionar el botón “Agregar” junto al producto deseado; el producto aparece en el carrito.
- Eliminar del carrito: en la sección del carrito, presionar “Eliminar” junto al producto que se quiere quitar.



12. Bitácora de desarrollo

Registro de avances, decisiones, dificultades, fuentes consultadas y soluciones aplicadas. Debe actualizarse durante el semestre y no completarse únicamente al final.

Semana / fecha	Avances
Semana 1	Instalé VirtualBox, cree la VM Lubuntu e instalé Linux. Configure 6672 MB de RAM, 4 CPU y red NAT
Semana 2	Instalé GIT, Apache, PHP y MYSQL y verifique que los servicios estuvieras activos.
Semana 3	Cree la base de datos cc con las tablas de productos y carrito y su relación
Semana 4	Desarrolle el proyecto PHP con arquitectura MVC y configure el VirtualHost
Semana 5	Traslade el carrito PHP a versión JAVA
Semana 6	Probe los EndPoints de la API y el flujo de externo a extremo.